

Day 24 : App Optimization and Basic Practices

1. Introduction

Flutter applications work well by default, but as the application grows, poor coding practices can reduce performance.

Optimization helps to

- Reduce CPU usage
- Reduce memory usage
- Increase FPS
- Improve UI responsiveness
- Reduce APK size
- Increase battery efficiency
- Prepare the app for Play Store release

2. Flutter Performance Optimization

A. Using const Constructors

Using **const** prevents Flutter from rebuilding widgets unnecessarily.

Without const

```
Text("Welcome")
```

Optimized

```
const Text("Welcome")
```

3. Avoid Unnecessary Rebuilds

Instead of rebuilding the entire screen, rebuild only required widgets.

```
ValueNotifier<int> counter = ValueNotifier(0);

ValueListenableBuilder(
  valueListenable: counter,
  builder: (_, value, __) {
    return Text("$value");
  },
)
```

4. ListView.builder

For long lists always use

```
ListView.builder(  
  itemCount:1000,  
  itemBuilder:(context,index){  
    return ListTile(  
      title: Text("Item $index"),  
    );  
  },  
)
```

5. Image Caching

```
CachedNetworkImage(  
  imageUrl:  
    "https://picsum.photos/300",  
  placeholder: (context,url)=>  
    CircularProgressIndicator(),  
  errorWidget:  
    (context,url,error)=>Icon(Icons.error),  
)
```

Dependencies:

cached_network_image: ^3.3.1

6. Lazy Loading

Only loads new data when required.

```
if(index==items.length-1){  
  loadMore();  
}
```

7. App Size Optimization

A. Remove Unused Resources

Delete

Unused images

Unused fonts

Unused packages

Unused Dart files

- Compress Images
Instead of 2 MB image
Compress to 200 KB

B. Use SVG Images

Instead of PNG

```
SvgPicture.asset(  
  "assets/logo.svg",  
);
```

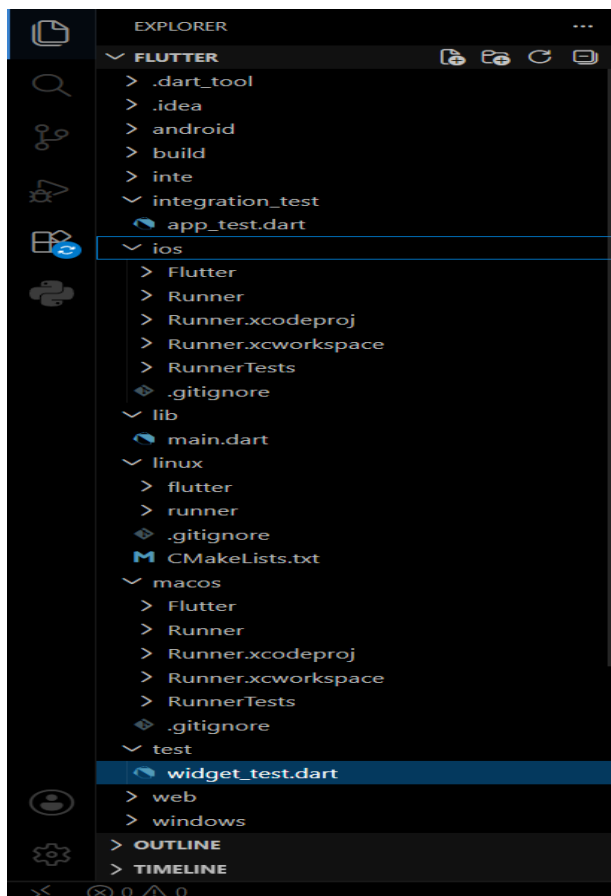
Dependency :

flutter_svg: ^2.0.10

C. Analyze App Bundle

```
flutter build appbundle  
  
flutter analyze  
  
flutter pub deps
```

8. Code Organization



9. Constants File

```
class AppStrings{

  static const appName="Optimization Demo";

  static const welcome="Welcome";

}
```

10. Theme Files

```
final ThemeData appTheme=ThemeData(

  primarySwatch:Colors.green,

  useMaterial3:true,

);
```

11. Flutter Best Practices

1. Error Handling

```
try{

  fetchData();

}catch(e){

  print(e);

}
```

2. Loading State

```
isLoading

?

CircularProgressIndicator()

:

Text("Loaded")
```

3. Dispose Controllers

```
@override  
  
void dispose(){  
  
  controller.dispose();  
  
  super.dispose();  
  
}
```

4. Accessibility

```
Semantics(  
  
  label:"Login Button",  
  
  child:ElevatedButton(  
  
    onPressed:(){},  
  
    child:Text("Login"),  
  
  ),  
  
)
```